Învățare Automată

Detecția (numărarea) obiectelor în imagini cu semnale radio

Smărăndoiu Eduard-Ştefan · grupa 231

1. Problema şi datele

Tema proiectului este să antrenez un model care, dată fiind o imagine cu un semnal radio, să spună în care din cele cinci clase se încadrează. Setul de antrenare are 15.500 de imagini, iar cel de test 5.500, fiecare în format PNG, cu eticheta dată într-un fişier `.csv` separat. Am respectat de la început restricția cea mai importantă a competiției: nu am folosit date externe şi niciun model predefinit sau preantrenat. Tot ce urmează, inclusiv rețeaua convoluțională, este construit de la zero, cu numpy, scikit-learn (doar pentru SVM şi Naive Bayes) şi PyTorch ca bibliotecă de calcul tensorial.

Înainte de a alege un model, m-am uitat efectiv la imagini ca să înțeleg ce le deosebește. Mi-au confirmat ce sugerează şi numele claselor: **clasa este de fapt numărul de obiecte (linii de semnal) din imagine**. Figura 1 arată câte un exemplu din fiecare clasă. Se văd nişte dungi verticale luminoase pe un fundal plin de zgomot. Fiecare dungă este un semnal, iar pe măsură ce urcăm de la clasa 1 spre clasa 5 apar tot mai multe. Clasa 1 are de obicei o singură linie pe un fundal foarte zgomotos, clasele de mijloc au două-trei, iar la clasa 4 apar şi nişte benzi orizontale în plus.

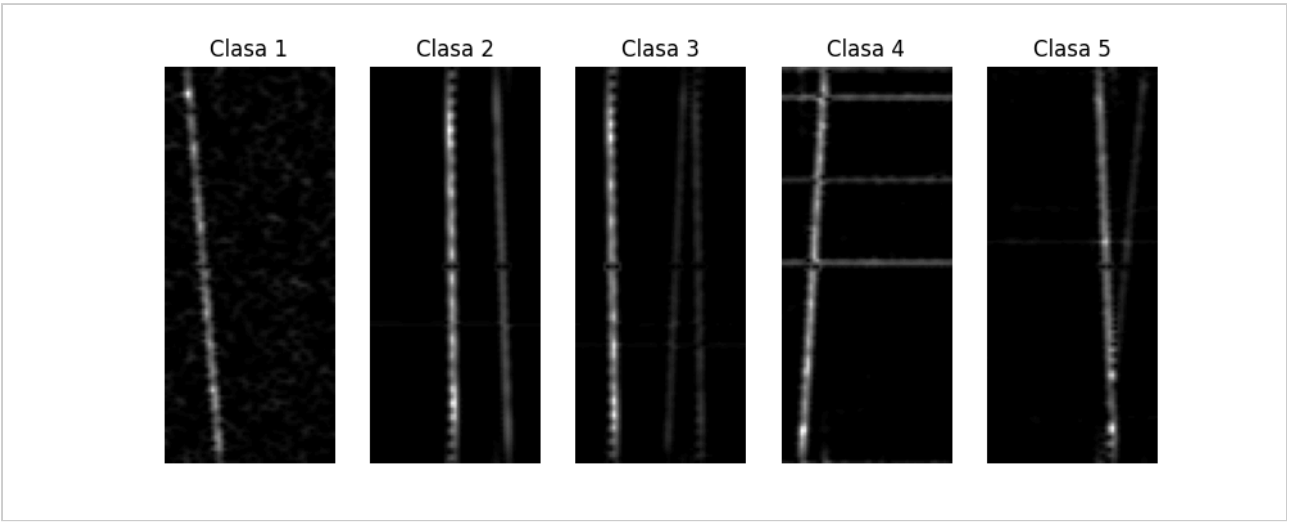


Figura 1. Câte un exemplu reprezentativ din fiecare clasă. Numărul de dungi verticale creşte cu indicele clasei.

Distribuția claselor (Figura 2) este aproape echilibrată: clasa 1 are în jur de 3.500 de exemple, iar celelalte patru cam 3.000 fiecare. Diferența e mică, dar mai târziu am observat că modelele slabe profită de ea şi prezic clasa 1 mult mai des decât ar trebui, pentru că e cea mai sigură variantă „din lene".

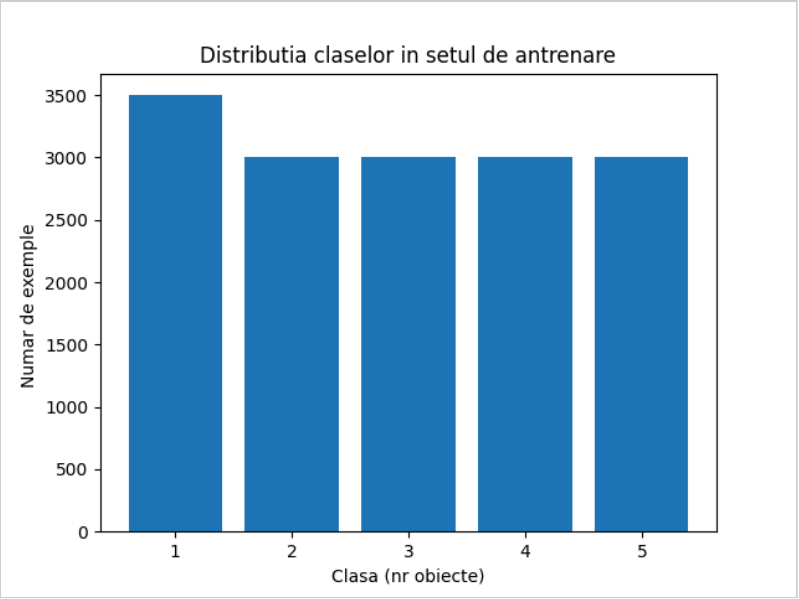


Figura 2. Numărul de exemple pe fiecare clasă în setul de antrenare.

Cea mai utilă imagine pentru deciziile de mai târziu a fost media tuturor imaginilor dintr-o clasă (Figura 3). Surpriza a fost că **mediile celor cinci clase sunt practic identice**: toate arată ca un dreptunghi gri uniform, cu o aceeași linie neagră orizontală pe la mijloc (un artefact prezent în toate imaginile). Cu alte cuvinte, dacă te uiți doar la intensitatea medie a pixelilor, clasele nu se disting deloc. Asta mi-a spus din start că orice metodă care compară imaginile la nivel de pixel brut (cum face kNN sau Naive Bayes pe pixeli) va eșua, fiindcă informația care contează nu e în luminozitatea generală, ci în structura fină: câte linii sunt și cum sunt aranjate.

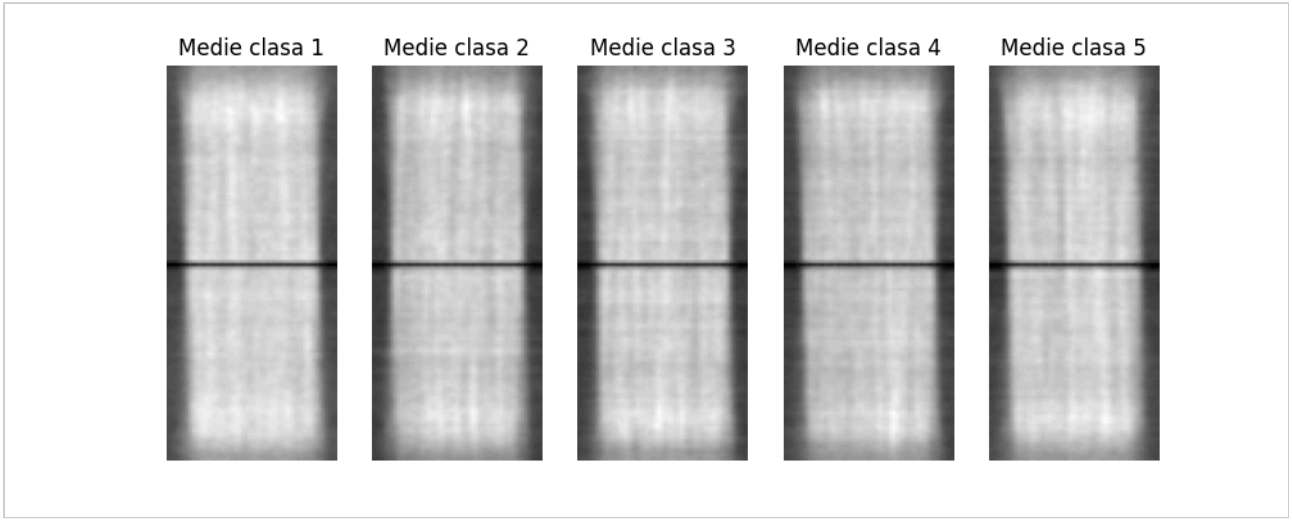


Figura 3. Imaginea medie pe fiecare clasă. Faptul că sunt aproape identice arată că diferența dintre clase nu stă în pixelii bruți, ci în detalii.

2. Preprocesare și augmentare

Toate imaginile au aceeași dimensiune, 128 pe înălțime și 55 pe lățime, și le-am încărcat pe tonuri de gri (un singur canal). Pentru partea clasică le-am întins într-un vector, iar pentru rețea le-am ținut ca matrice 2D ca să pot folosi convoluții. Singura normalizare la modelele clasice a fost `StandardScaler` înainte de

SVM, fiindcă SVM-ul cu kernel RBF e sensibil la scara trăsăturilor. La rețea am normalizat fiecare fold cu media și deviația lui standard (z-score), calculate doar pe datele de antrenare ale foldului ca să nu „scurg" informație din validare.

Pentru rețea am adăugat și augmentare, adică transformări aleatoare aplicate imaginilor în timpul antrenării ca modelul să nu memoreze exemplele:

- **Translație verticală** de până la 5%. Am ales doar pe verticală și doar puțin pentru că semnalul poate să apară ușor decalat de sus în jos de la o imagine la alta, dar nu vreau să mut atât de mult încât să tai linii importante.
- **Mixup** cu alpha 0.3, adică amestec două imagini și etichetele lor cu un raport aleator. Forțează rețeaua să nu fie sigură pe un singur exemplu și ajută la generalizare.
- **Random Erasing (cutout)** cu probabilitate 0.5, care șterge un dreptunghi mic din imagine. Astfel rețeaua nu se poate baza pe o singură porțiune din semnal, ci trebuie să folosească toată imaginea.

3. Clasificare sau regresie?

O întrebare pe care chiar o ridică enunțul este dacă problema asta ar trebui privită ca o clasificare sau ca o regresie, de vreme ce clasele sunt de fapt un număr de obiecte. Merită discutată pentru că răspunsul nu e evident.

Dacă o tratez ca **regresie**, modelul ar prezice un număr (de exemplu 3,4) pe care apoi îl rotunjesc la o clasă. Avantajul ar fi că modelul ar „ști" că 4 e între 3 și 5, deci o greșeală cu o clasă ar fi penalizată mai puțin decât una cu trei clase, ceea ce se potrivește cu felul în care e construită problema. Dezavantajul e că ieșirea trebuie discretizată înapoi, iar metrica de evaluare oricum se uită la clasa exactă, nu la cât de aproape ai fost.

Eu am ales **clasificarea**, din două motive practice: se potrivește direct cu metrica (acuratețe pe cinci clase) și e mai simplu de antrenat cu un strat softmax la final. Dezavantajul teoretic al clasificării e că un softmax obișnuit tratează toate greșelile la fel; pentru el, a confunda 4 cu 5 e la fel de grav ca a confunda 4 cu 1, deși în realitatea problemei nu e. Interesant e însă că, deși nu i-am spus rețelei nimic despre ordinea claselor, ea a învățat-o singură: dacă te uiți la matricea de confuzie din secțiunea 6, aproape toate greșelile cad pe clasele vecine. Asta e cel mai bun argument că alegerea clasificării a fost suficientă, fiindcă rețeaua a prins implicit noțiunea de cantitate. Ca direcție de îmbunătățire, o variantă intermediară ar fi clasificarea ordinală, care codifică explicit ordinea claselor; n-am folosit-o aici, dar ar fi pasul logic următor.

4. Abordările clasice

Am început cu modele simple ca să am un punct de referință și ca să înțeleg cât de greu e de fapt task-ul. Toate folosesc aceeași împărțire 80/20 train/validare, cu seed fix (42) ca rezultatele să fie reproductibile.

4.1 Naive Bayes

Primul model a fost un Naive Bayes multinomial, atât pe pixelii bruți cât și pe trăsăturile LBP (descrise mai jos), discretizând valorile în câteva intervale. Rezultatele (Tabel 4) au fost dezamăgitoare în ambele cazuri, în jur de 0,22, adică abia peste nivelul de a ghici (cu cinci clase, ghicitul dă 0,20). Numărul de intervale nu

a schimbat aproape nimic. Era de așteptat: Naive Bayes presupune că trăsăturile sunt independente între ele, ceea ce pentru pixeli vecini într-o imagine e complet fals, pentru că pixelii unei linii sunt puternic corelați. Matricea de confuzie (Figura 4) arată un model care împrăștie predicțiile haotic și are o preferință nejustificată pentru clasele 1 și 5.

nr. bins	NB pe LBP	NB pe pixeli bruți
3	0.2265	0.2274
5	0.2265	0.2039
7	0.2265	0.2068
9	0.2265	0.2119
11	0.2265	0.2055

Tabel 4. Naive Bayes multinomial, cu discretizarea trăsăturilor în diferite numere de intervale.

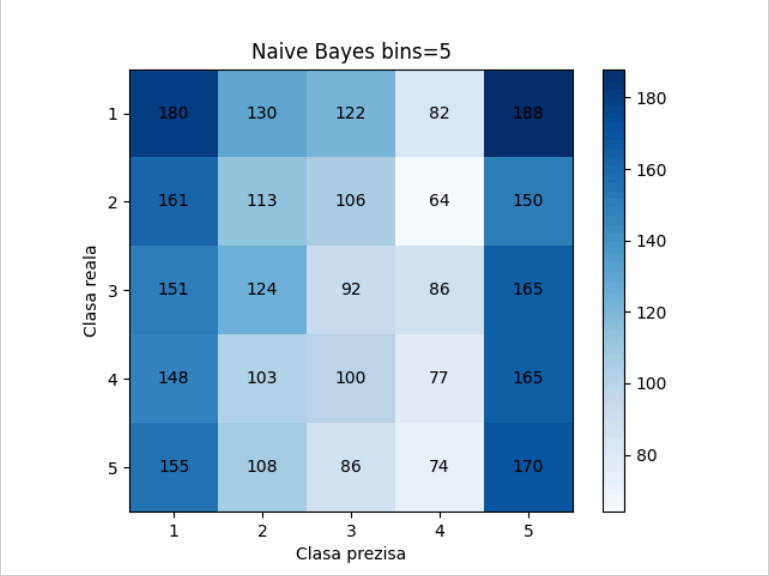


Figura 4. Matricea de confuzie pentru Naive Bayes (5 intervale). Predicțiile sunt împrăștiate, fără o diagonală clară.

4.2 k-Nearest Neighbors

Am scris kNN de la zero, calculând distanțele cu numpy și votând clasa majoritară între cei mai apropiați k vecini. Am testat ambele distanțe din curs, L1 (Manhattan) și L2 (Euclidiană), și mai multe valori pentru k (Tabel 3). Pe trăsăturile LBP a mers ceva mai bine decât Naive Bayes, cel mai bun fiind k=7 cu L1, la 0,37. Pe pixeli bruți a fost și mai slab, sub 0,21, fix din motivul anticipat la imaginea medie: două imagini din clase diferite pot avea pixeli foarte asemănători, deci „cel mai apropiat vecin" nu înseamnă „aceeași clasă". kNN suferă aici și de blestemul dimensionalității: cu mii de trăsături, noțiunea de distanță devine puțin informativă. Matricea de confuzie (Figura 5) arată că modelul trage majoritatea predicțiilor spre clasa 1.

k	L1 (Manhattan)	L2 (Euclidiană)
1	0.334	0.324
3	0.340	0.326
5	0.354	0.320
7	**0.370**	**0.344**
9	0.356	0.340

Tabel 3. kNN pe trăsături LBP, variind k și metrica de distanță (subset de 3000 train / 500 validare).

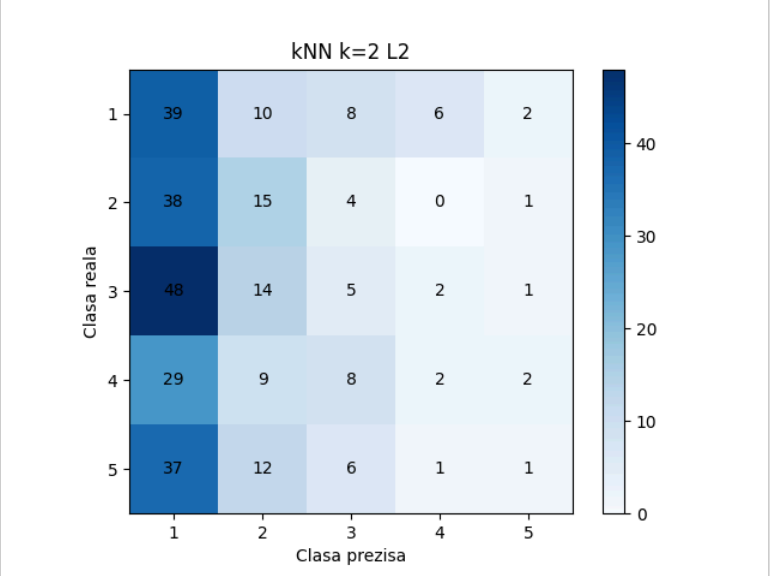


Figura 5. Matricea de confuzie pentru kNN (k=2, L2) pe un subset de validare. Coloana clasei 1 e supra-populată.

4.3 LBP + SVM

Aici a fost primul progres real. În loc să dau modelului pixeli, am extras întâi trăsături **LBP (Local Binary Patterns)**, scrise tot de la zero. Ideea LBP este simplă: pentru fiecare pixel, compar intensitatea lui cu a celor opt vecini și formez un cod de 8 biți care descrie textura locală din jurul lui. Apoi fac o histogramă a acestor coduri pe toată imaginea. Ca să prind texturi la scări diferite, am calculat LBP la cinci raze (de la 1 la 5) și am lipit histogramele cap la cap, rezultând un vector de 1280 de valori per imagine.

Peste aceste trăsături am antrenat un SVM cu kernel RBF, cu datele normalizate. Am tunat parametrul de regularizare C și, separat, am observat un efect important al lui gamma (Tabel 1 și 2). Trecerea de la gamma implicit (scale) la 1,5 ori valoarea implicită a adus un salt de aproape 6 puncte pe tot intervalul. Cea mai bună configurație a fost **C=5 cu gamma de 1,5 ori valoarea implicită, la 0,531**. SVM-ul pe LBP a urcat deci de la nivelul de ghicit până la peste 53%, ceea ce confirmă că textura locală chiar conține informație utilă, dar și că se atinge un plafon. LBP descrie cum arată textura, nu numără obiecte, iar numărarea e exact ce cere problema.

C	Acuratețe validare
1	0.5219
5	**0.5310**
10	0.5252
50	0.5048
100	0.4952

Tabel 1. Tunarea parametrului C pentru SVM cu kernel RBF (gamma = 1.5 ori scale), pe trăsături LBP.

C	gamma = scale (x1)	gamma = scale x1.5
1	0.4690	0.5219
5	0.4703	**0.5310**
10	0.4729	0.5252
50	0.4658	0.5048
100	0.4684	0.4952

Tabel 2. Efectul lui gamma asupra SVM-ului. Creșterea lui gamma de la valoarea implicită scale la 1.5 ori valoarea ei aduce un salt de aproape 6 puncte procentuale pe tot intervalul de C.

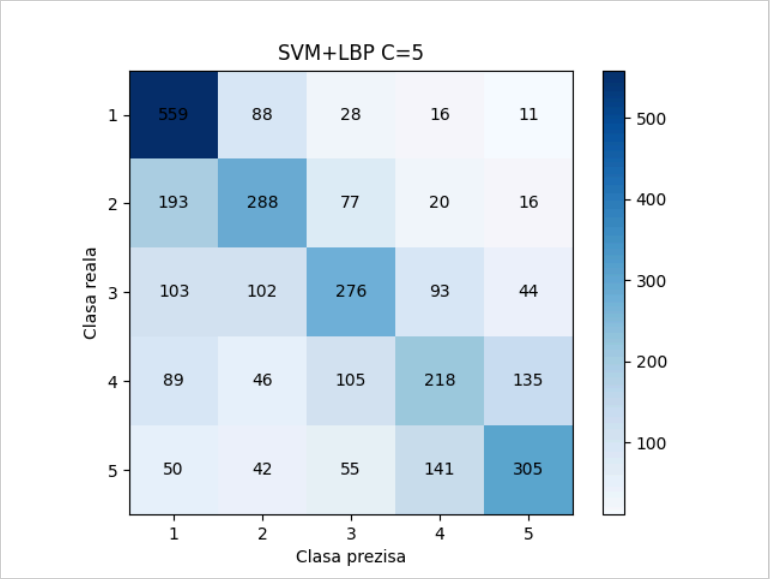


Figura 6. Matricea de confuzie pentru SVM+LBP (C=5). Diagonala începe să se contureze, dar confuziile cu clasa 1 și între clasele vecine rămân mari.

Matricea SVM (Figura 6) e prima cu o diagonală vizibilă. Recall-ul pe clasa 1 e bun (0,80), dar scade serios pe clasele cu mai multe obiecte (Tabel 6). Concluzia după partea clasică a fost clară: cu trăsături făcute de mână ajung undeva la 50%, dar ca să trec peste am nevoie de un model care să învețe singur ce să caute în imagine.

5. Rețeaua convoluțională

Pentru partea principală am construit o rețea convoluțională de la zero. Spre deosebire de LBP, unde eu decid ce trăsături să extrag, rețeaua își învață singură filtrele direct din date. Am pornit de la o structură de tip rețea reziduală, dar am adaptat-o la specificul imaginilor mele.

5.1 Arhitectura

Câteva decizii de proiectare merită explicate, pentru că nu sunt arbitrare:

- **Filtru de intrare asimetric (5x3).** Imaginile mele sunt mult mai înalte (128) decât late (55). Un filtru mai înalt decât lat „vede" mai mult pe direcția în care e mai multă informație, în loc să folosesc un pătrat care ar trata cele două axe la fel.
- **Blocuri reziduale.** Fiecare bloc are o scurtătură care adună intrarea peste ieșirea procesată. Fără ea, rețelele adânci se antrenează prost, pentru că gradientul se pierde pe drum; scurtătura îi lasă o cale directă înapoi și împiedică degradarea.
- **Atenție pe canale (squeeze-and-excitation).** După fiecare bloc, recalibrez canalele: rețeaua învață o pondere între 0 și 1 pentru fiecare canal și le înmulțește. Practic, dacă un filtru prinde doar zgomot de fond, primește o pondere aproape de zero și e ignorat. Pe imagini atât de zgomotoase ca ale mele, asta e exact ce trebuie, adică un mecanism care să distingă canalele utile de cele de zgomot.

Corpul rețelei alternează blocuri care păstrează dimensiunea cu blocuri care o înjumătățesc (stride 2), crescând treptat numărul de canale de la 32 la 256. La final, o agregare globală reduce fiecare canal la un număr și un strat liniar dă cele cinci scoruri de clasă.

5.2 Antrenarea

Pentru antrenare am folosit funcția de pierdere cross-entropy cu **label smoothing** (0,05), care descurajează rețeaua să devină exagerat de sigură pe o predicție, util când datele au mult zgomot, fiindcă un model prea încrezător se înșală cu convingere. Optimizatorul a fost AdamW cu weight decay pentru regularizare, iar rata de învățare a urmat o planificare OneCycle (crește la început, apoi scade lent). Am antrenat în precizie mixtă (AMP) ca să încapă pe GPU și să meargă mai repede, cu tăierea gradientului ca să nu explodeze.

În loc de o singură împărțire train/validare, am folosit **validare încrucișată stratificată pe 5 folduri**. Asta înseamnă că antrenez cinci modele, fiecare pe 4/5 din date și validat pe a cincea parte, astfel încât fiecare imagine ajunge exact o dată în validare. Acuratețea raportată (OOF, „out-of-fold") e calculată pe aceste predicții de validare adunate de pe toate foldurile, deci pe tot setul de 15.500 de imagini, fără ca vreun model să-și fi văzut vreodată propriile date de validare.

5.3 TTA și combinarea modelelor

La predicție am folosit două tehnici care au mai adăugat câteva puncte. Prima e **TTA (test-time augmentation)**: rulez aceeași imagine deplasată în mai multe direcții și mediez predicțiile, dând cea mai mare pondere imaginii nemișcate din centru. O predicție mediată pe variante mici e mai stabilă decât una singură. A doua e **combinarea celor cinci modele** de la cross-validation: adun probabilitățile lor și aleg clasa cu suma cea mai mare. Cum fiecare fold a învățat ceva ușor diferit, media lor greșește mai rar decât oricare model luat separat.

Configurație	Acuratețe OOF
4 folduri, 35 epoci, fără TTA	0.736
4 folduri, 40 epoci, TTA 5 direcții (pondere egală)	0.742
5 folduri, 55 epoci, TTA ponderat 9 direcții	**0.753**

Tabel 5. Evoluția acurateței CNN pe măsură ce am adăugat folduri, epoci și o schemă de TTA mai bună.

Tabelul 5 arată cum am ajuns la rezultatul final. Pornind de la o configurație de bază la 0,736, adăugarea unui TTA mai bun și a încă unui fold plus mai multe epoci au dus acuratețea OOF la **0,753**. Pe leaderboard-ul public al competiției, această abordare a obținut un scor de **0,78**.

6. Matricile de confuzie și ce ne spun

Figura 7 e matricea de confuzie a rețelei finale, calculată pe toate cele 15.500 de imagini prin predicțiile out-of-fold. Diagonala e clar dominantă, dar mai interesant e **unde** greșește modelul.

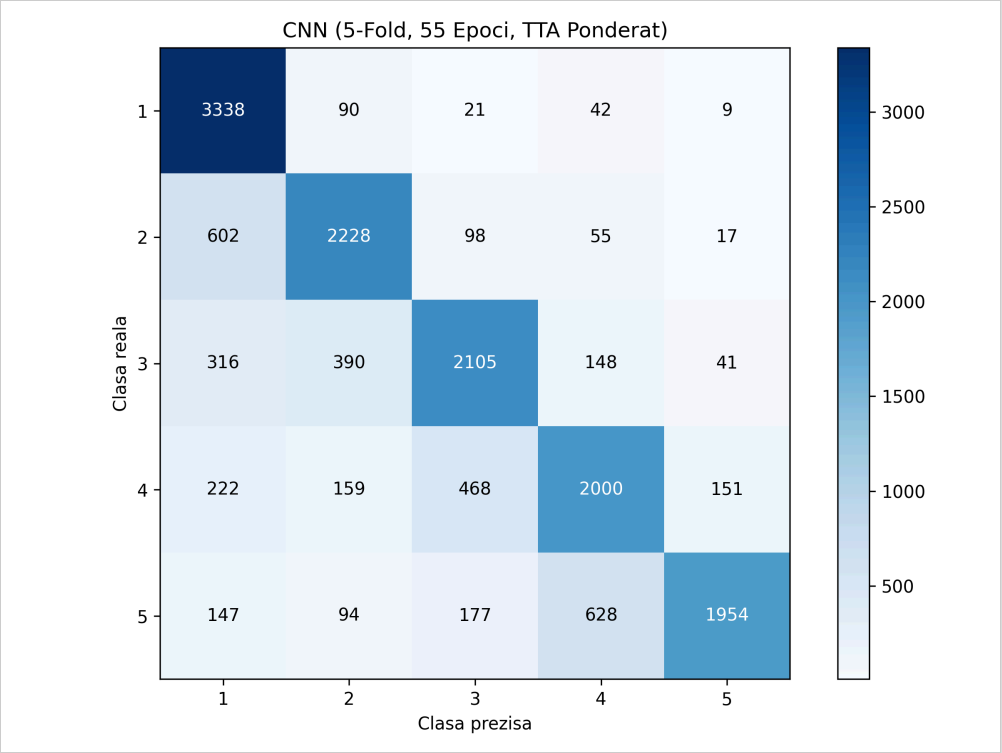


Figura 7. Matricea de confuzie a rețelei finale (5 folduri, 55 epoci, TTA ponderat), pe tot setul de antrenare prin predicții out-of-fold.

Două lucruri ies în evidență. Primul: recall-ul scade constant pe măsură ce cresc clasele: 0,95 pe clasa 1, apoi 0,74, 0,70, 0,67 și 0,65 pe clasa 5 (Tabel 6). Are sens: e ușor să detectezi un singur obiect pe zgomot, dar cu cât sunt mai multe linii care se pot suprapune și amesteca cu zgomotul, cu atât e mai greu să le numeri exact. Al doilea, și cel mai relevant pentru discuția despre clasificare versus regresie: **aproape toate greșelile cad pe clasele vecine**. O imagine din clasa 5 e confundată cel mai des cu clasa 4 (628 de cazuri), aproape niciodată cu clasa 1 (147). La fel, clasa 2 e confundată mai ales cu 1, clasa 4 cu 3. Modelul nu face greșeli „sărind" peste clase. Asta confirmă că, deși am antrenat un clasificator obișnuit, el a învățat singur structura ordinală a problemei, exact argumentul din secțiunea 3.

Comparativ, matricea SVM+LBP (Figura 6) are aceeași tendință generală, dar mult mai slabă: diagonala e prezentă, însă confuziile cu clasa 1 sunt mult mai mari, iar recall-ul pe clasele de mijloc abia trece de 0,40. Diferența dintre cele două arată concret ce a adus rețeaua: nu doar o diagonală mai puternică, ci capacitatea de a nu mai „fugi" spre clasa 1 când nu e sigură.

Clasă	Recall SVM+LBP	Recall CNN
1 (un obiect)	0.80	0.95
2	0.49	0.74
3	0.45	0.70
4	0.37	0.67
5 (cinci obiecte)	0.51	0.65

Tabel 6. Recall pe fiecare clasă. La ambele modele se vede aceeași tendință: cu cât sunt mai multe obiecte de numărat, cu atât scade acuratețea.

7. Concluzii

Privind tot drumul, progresul e clar și are o explicație la fiecare pas. Modelele care se uită la pixeli bruți (Naive Bayes, kNN pe pixeli) rămân la nivelul de ghicit, în jur de 20%, fiindcă informația care contează nu e în intensitatea pixelilor, ci în structura imaginii, lucru pe care l-am văzut încă de la imaginea medie. Trecerea la trăsături de textură făcute de mână (LBP) plus un SVM a ridicat performanța la peste 53%, dovedind că textura locală chiar conține semnal, dar și că o reprezentare construită manual are un plafon, pentru că descrie textura fără să numere obiecte. Rețeaua convoluțională, care își învață singură filtrele și are un mecanism de atenție ca să suprime zgomotul, a sărit la 75% pe validare și 78% pe leaderboard-ul public.

Dacă ar fi să continui, primele direcții ar fi clasificarea ordinală (care s-ar potrivi cu structura clar ordinală observată în matricea de confuzie) și o atenție specială pe clasele mari, unde modelul încă pierde cele mai multe puncte din cauza confuziilor cu vecinii apropiați.